Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет <факультет>

Кафедра <кафедра>

Дисциплина: Тестирование программного обеспечения

ОТЧЁТ
по лабораторной работе
на тему

РАЗРАБОТКА ТЕСТ-КЕЙСОВ

Студент: <И.О. Фамилия студента>

Руководитель: <И.О. Фамилия руководителя>

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Изучить структуру тест-кейса, освоить эквивалентное разбиение и анализ граничных значений, оформить и выполнить тест-кейсы для формы заявки на раннее подключение веб-приложения DSLKit.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Теория дана по книге С. С. Куликова «Тестирование программного обеспечения. Базовый курс» (EPAM Systems, 2015–2025), главы 2.4 и 2.7.2.

Тест-кейс — набор входных данных, условий выполнения и ожидаемых результатов, разработанный для проверки определённого свойства или поведения программы. Если не указаны входные данные, условия или ожидаемый результат, либо неясна цель, тест-кейс считается плохим. Тест-кейсы обеспечивают повторяемость проверок, передачу знаний в команде, регрессионное тестирование и улучшение требований.

Класс эквивалентности — набор данных, обрабатываемых одинаково и дающих одинаковый результат. Граничное условие — значение на границе классов эквивалентности. Для диапазона проверяют последнее допустимое и первое недопустимое значения, а также «опасные» значения вроде нуля и пустой строки. Позитивные тест-кейсы берут из допустимых классов, негативные из недопустимых.

3 ХОД РАБОТЫ

3.1 Выбор use case и требования

Тестируется вариант использования UC-01, описанный ниже. Проверку полей выполняет функция validateWaitlist (файл site/js/utils/validation.js), которая возвращает список неверных полей. Проект развёрнут локально командой npm start.

Название.

– **UC-01.** Подать заявку в лист ожидания.

Система и модуль.

– **Система:** DSLKit — *редактор диаграмм как кода (статический сайт,* site/*);

– **Модуль:** «Лист ожидания беты» — форма на главной странице;

– **Ссылка:** http://localhost:5173/#waitlist.

Основной актор.

– Посетитель сайта (без регистрации и авторизации).

Цель.

– Оставить контакты, чтобы получить приглашение в бету DSLKit.

Предусловия.

– 1. Открыта главная страница (*/* или */en/*);

– 2. Форма пуста: нотация не выбрана, размер команды 0, согласие не отмечено.

Основной поток.

– 1. Посетитель нажимает ***«В бету» в шапке (или «Записаться в бету») и попадает к форме** #waitlist*;

– 2. Вводит ***«Имя\»* и «Электронная почта\»****;

– 3. Выбирает ***«Нотация\»**** (IDEF0, DFD, ERD, BPMN, UML, «Сети Петри»);

– 4. Вводит ***«Размер команды\»* (0–500) и при желании «Комментарий»***;

– 5. Отмечает согласие\ *и нажимает «Отправить»**;

– 6. Система проверяет поля по таблице «Структура данных» (*validateWaitlist*);

– 7. Система сохраняет заявку в *LocalStorage* (ключ *dslkit.waitlist*);

– 8. Система открывает страницу благодарности *thanks.html*;

– * — обязательное поле.

Альтернативные потоки.

– ***A1.*** Поле неверно (шаг 6): под полем красное сообщение, фокус на первом неверном поле, заявка не сохраняется; после исправления сообщение исчезает;

– ***A2.*** Повторная заявка с той же почтой: принимается как новая запись (дефект D-1);

– *A3.* Посетитель закрыл страницу до шага 5: данные не сохраняются.

Постусловия.

– Успех: в *LocalStorage* на одну запись больше, открыта *thanks.html*. Ошибка проверки: записей не добавилось, значения остаются в полях.

Исключительные ситуации.

– *E1. Запись в* LocalStorage *не удалась: страница благодарности всё равно открывается, заявка потеряна*, ошибки нет (дефект D-2).

Другие точки входа.

– Ссылки «Записаться в бету: UML / BPMN / …» в разделе нотаций, кнопка «В бету» на *privacy.html* и *thanks.html*, адрес */en/#waitlist*.

Связанные задачи.

– *R4* — Форма заявки и валидация;

– *UC-00* — Прочитать политику конфиденциальности;

– *UC-03* — Авторизация (в текущей версии не реализована);

– *UC-04* — Запись в бету из React-приложения;

– *TC-WL-001…022* — Тест-кейсы формы (лаб. 2);

– *SM-07, CP-35, CP-36* — Проверки чек-листов (лаб. 3).

Скриншоты.

– *SCR-01* — Форма: ошибки при пустой отправке;

– *SCR-02* — Форма: заполнена;

– *SCR-03* — Страница благодарности.

Реализация.

– *site/index.html* (форма *#waitlist-form*), *site/js/components/waitlist-form.js*, *site/js/utils/validation.js*, *site/js/utils/storage.js*, *site/thanks.html*.

Ссылки.

– http://localhost:5173/#waitlist;

– http://localhost:5173/thanks.html;

– https://github.com/KsushaKhadzhinova/DSLKit-Project.

Таблица 3.1 – Структура данных заявки (сущность WaitlistEntry)

Поле	Тип	Длина	Обязат.	Уник.	Маска	Связь
name* (П1)	string	2–60	Да	Нет	буквы, пробел, апостроф, дефис; первая — буква	—
email* (П2)	string	6–120	Да	Нет (дубликаты принимаются, D-1)	name@domain.zone	

Поле	Тип	Длина	Обязат.	Уник.	Маска	Связь
notation* (П3)	enum	3–5	Да	Нет	idef0, dfd, erd, bpmn, uml, petri	→ спр вочни Notati N:1
team* (П4)	integer	0–500	Да	Нет	целое число; по умолча-нию 0	—
comment (П5)	string	0–500	Нет	Нет	без маски	—
consent* (П6)	boolean	—	Да, только true	—	флажок; в запись не сохраня-ется	→ полит (priva
createdAt	string ISO 8601	24	Да, ставит система	Нет	ГГГГ-ММ-ДДТЧЧ:ММ	— :CC.cc

Хранилище: массив JSON в LocalStorage, ключ dslkit.waitlist. Первичного ключа нет. Звёздочка означает обязательное поле.

3.2 Диаграмма и скриншоты

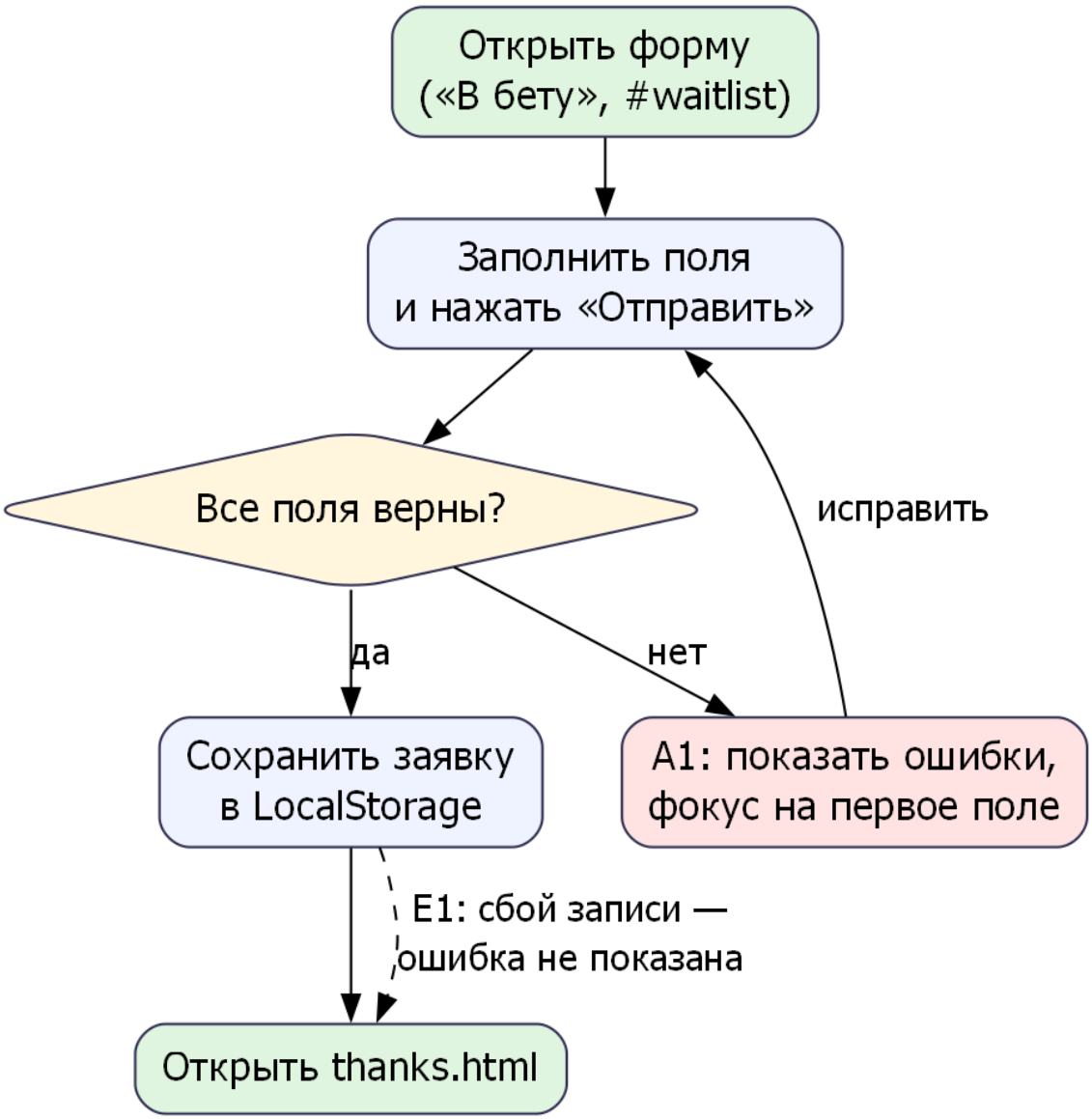


Рисунок 3.1 – Блок-схема сценария UC-01

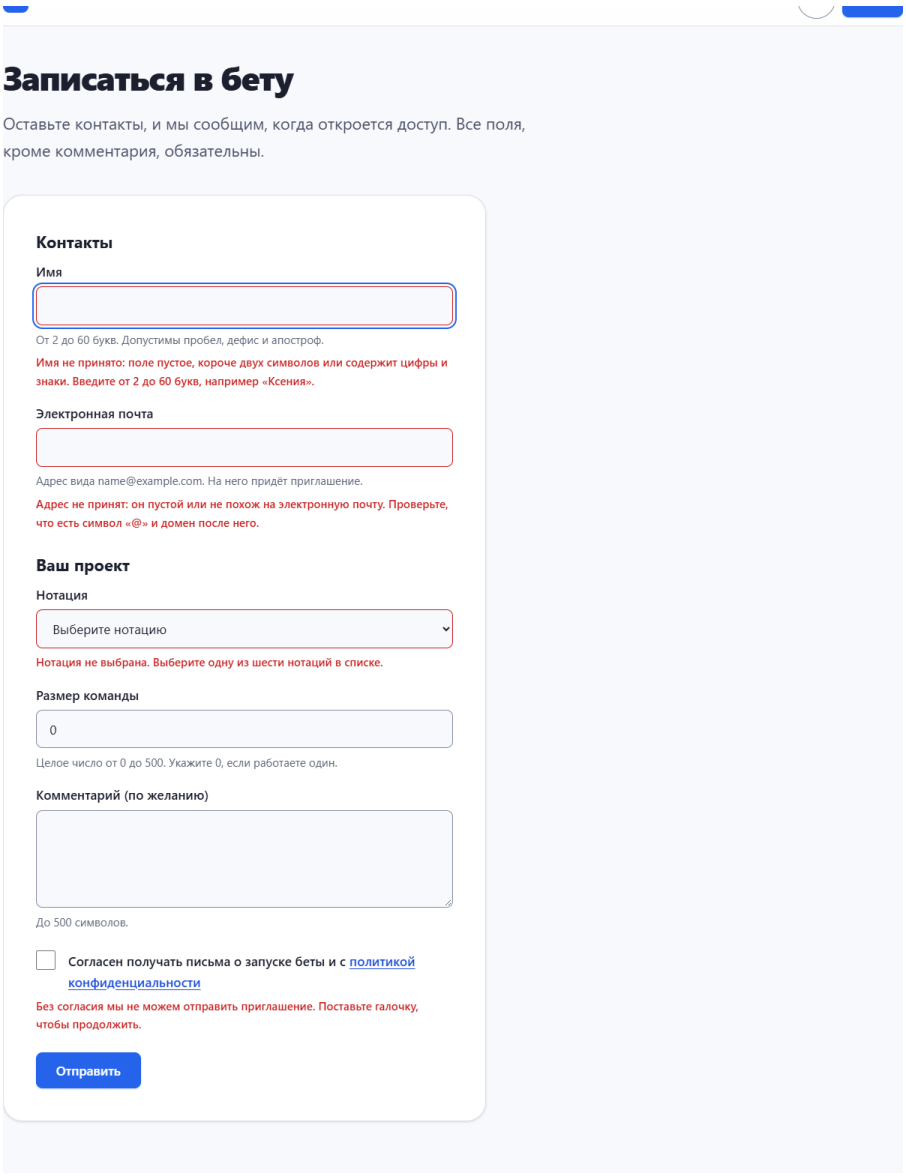


Рисунок 3.2 – Форма: ошибки при пустой отправке

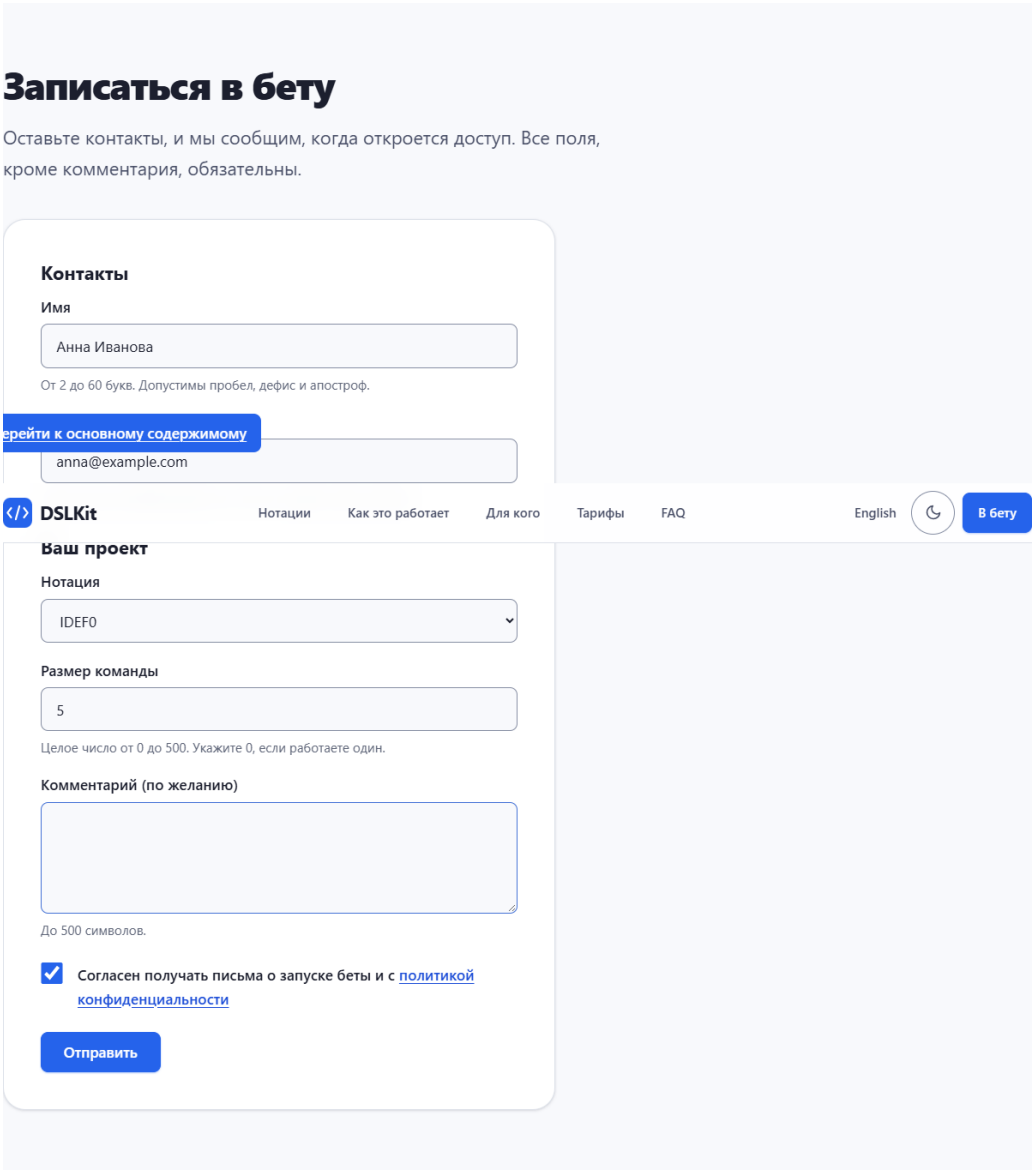


Рисунок 3.3 – Форма заполнена корректными данными

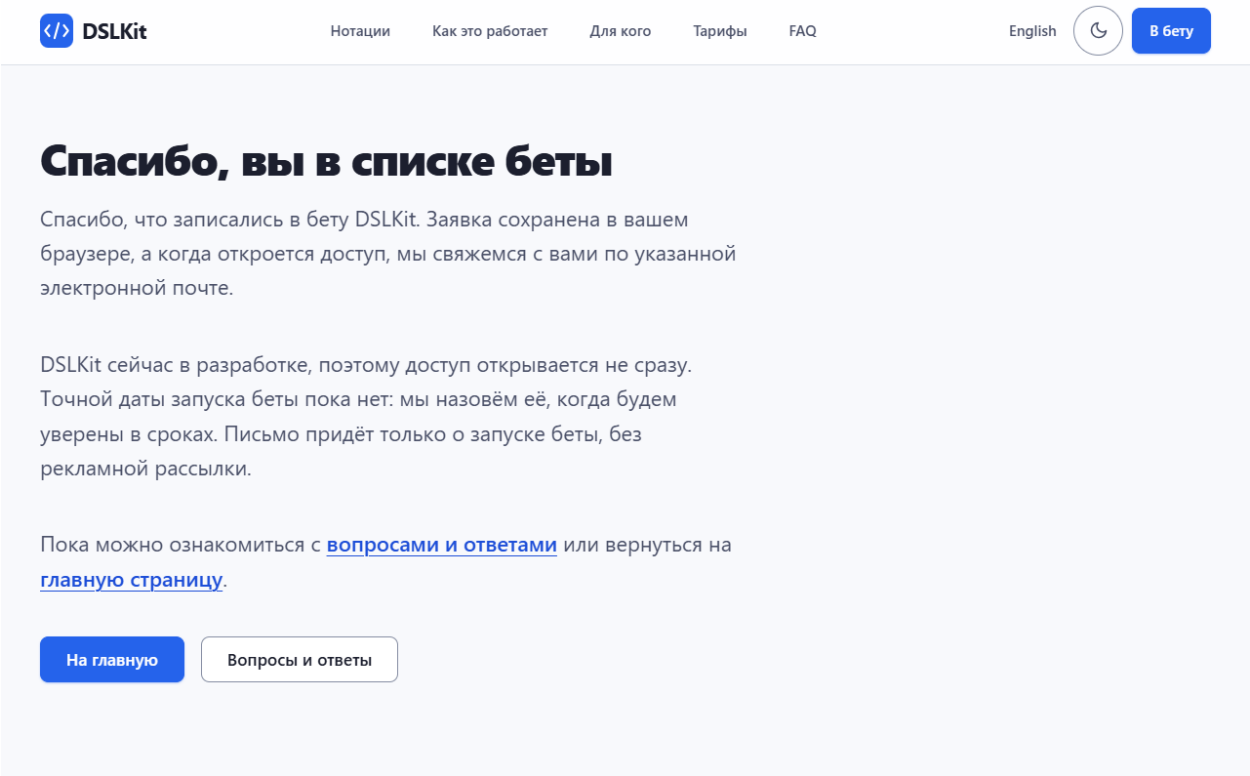


Рисунок 3.4 – Страница благодарности

Трассировка: правило П1 проверяют TC-WL-002…007, П2 008…011, П3 018, П4 012…017, П5 019 и 020, П6 021, поток А1 022.

Требования к полям приведены в таблице ниже.

Таблица 3.2 – Требования к полям формы

Поле	Правило
name	Начинается с буквы; буквы, пробел, апостроф, дефис; длина от 2 до 60
email	Формат адреса с доменом верхнего уровня; длина не более 120
notation	Одно из: idef0, dfd, erd, bpmn, uml, petri
team	Целое число от 0 до 500 (допустима строка из цифр)
comment	Не более 500 символов, необязательное
consent	Только true

3.3 Классы эквивалентности и границы

Для каждого поля выделены допустимый и недопустимые классы; границы проверяются парами «последнее допустимое и первое недопустимое» значение (таблица ниже).

Таблица 3.3 – Классы эквивалентности и граничные значения

Поле	Допустимые	Недопустимые	Границы
name	Буквы 2–60	Цифры, 1 символ, более 60	1 / 2 / 60 / 61
email	Корректный адрес	Без @, без домена, более 120	120 / 121
team	0–500	Отрицательное, больше 500, не число	-1 / 0 / 500 / 501
notation	Из списка	Любое другое значение	—
comment	0–500	Более 500	500 / 501
consent	true	false, отсутствует	—

3.4 Тест-кейсы и результаты

Разработано 22 тест-кейсов: позитивные, негативные и граничные. Идентификаторы в таблице сокращены: полный вид TC-WL-NNN. В столбце «Данные» указаны только поля, отличающиеся от базового корректного набора (name=Анна Иванова, email=anna@example.com, notation=idef0, team=5, comment пустой, consent=true). Общие предусловия: проект запущен, форма открыта, поля пусты. Постусловие: данные не сохраняются, состояние системы не меняется. Шаги общие: 1) подать набор данных на вход проверки формы; 2) получить список неверных полей. Ожидаемый результат указан в таблице.

Таблица 3.4 – Тест-кейсы и результаты выполнения

ID	Тип, приоритет	Заголовок	Данные	Ожидаемый	Фактичес
WL-001	позитивный, High	Все поля корректны	все поля корректны	ошибок нет	ошибок не (Passed)
WL-002	граничный, High	Имя из 1 символа (ниже границы 2)	name=”A”	name	name (Passed)
WL-003	граничный, Medium	Имя из 2 символов (нижняя граница)	name=”Ян”	ошибок нет	ошибок не (Passed)

ID	Тип, приоритет	Заголовок	Данные	Ожидаемый	Фактичес
WL-004	граничный, Medium	Имя из 60 символов (верхняя граница)	name=aaaaa… (60)	ошибок нет	ошибок не (Passed)
WL-005	граничный, Medium	Имя из 61 символа (выше границы)	name=aaaaa… (61)	name	name (Passed)
WL-006	негативный, High	Имя с цифрой	name=”Anna1”	name	name (Passed)
WL-007	позитивный, Low	Имя с апострофом и дефисом	name=”O’Neil-Smith”	ошибок нет	ошибок не (Passed)
WL-008	негативный, High	Email без символа @	email=anna.… (16)	email	email (Passed)
WL-009	негативный, Medium	Email без домена верхнего уровня	email=”anna@example”	email	email (Passed)
WL-010	граничный, Medium	Email длиной 120 символов	email=aaaaa… (120)	ошибок нет	ошибок не (Passed)
WL-011	граничный, Medium	Email длиной 121 символ	email=aaaaa… (121)	email	email (Passed)
WL-012	граничный, High	Размер команды 0 (нижняя граница)	team=0	ошибок нет	ошибок не (Passed)
WL-013	граничный, High	Размер команды 500 (верхняя граница)	team=500	ошибок нет	ошибок не (Passed)
WL-014	граничный, High	Размер команды 501 (выше границы)	team=501	team	team (Passed)

ID	Тип, приоритет	Заголовок	Данные	Ожидаемый	Фактичес
WL-015	граничный, Medium	Размер команды -1 (ниже границы)	team=-1	team	team (Passed)
WL-016	негативный, Medium	Размер команды — не число	team=”abc”	team	team (Passed)
WL-017	позитивный, Low	Размер команды строкой ”12”	team=”12”	ошибок нет	ошибок не (Passed)
WL-018	негативный, High	Нотация вне списка	notation=”xyz”	notation	notation (Passed)
WL-019	граничный, Low	Комментарий 500 символов	comment=xxxxx (500)	ошибок нет	ошибок не (Passed)
WL-020	граничный, Low	Комментарий 501 символ	comment=xxxxx (501)	comment	comment (Passed)
WL-021	негативный, High	Согласие на обработку не дано	consent=false	consent	consent (Passed)
WL-022	негативный, High	Пустая форма	name=undefined; email=undefined; notation=undefined; team=undefined; comment=undefined; consent=undefined	name, email, notation, team, consent	name, ema notation, team, consent (Passed)

4 АНАЛИЗ РЕЗУЛЬТАТОВ

Выполнено 22 тест-кейсов, пройдено 22, не пройдено 0. Дефектов не выявлено: проверка полей соответствует требованиям на всех границах. По-путно исправлена ошибка самих тестов на этапе отладки (неверный формат тестовых данных), к продукту она отношения не имеет.

5 ВЫВОД

Освоены оформление тест-кейсов по шаблону, эквивалентное разбиение и анализ граничных значений. Тест-кейсы автоматизированы (команда npm run tc:lab2) и повторяются без участия человека.